

Designing Transparent and Adaptive Interfaces for AI Tutors: A Human–Computer Interaction Approach

Aaditya Borse

borsea@uci.edu

University of California, Irvine
Irvine, California, USA

Abstract

Recent advances in large language models have renewed interest in intelligent tutoring systems that support personalized learning. Yet, most existing platforms offer limited transparency or user adaptability. This paper presents **LEO**, an intelligent learning environment designed for students in grades 6–12 that integrates adaptive feedback with transparent interface design. LEO aims to bridge the gap between intelligent automation and learner understanding through explainable interactions and better guidance. The system demonstrates how explainable feedback, visual reasoning traces, and adaptive interfaces can improve learner engagement and comprehension across subject areas. Aligning human–computer interaction principles with explainable AI and learning analytics leads to LEO addressing three key gaps in prior systems: opaque adaptation, disjointed dashboards, and limited learner control.

Keywords

Artificial Intelligence in Education, Large Language Models, Intelligent Tutoring Systems, Learning Analytics, Personalized Learning, Reinforcement Learning, Educational Dashboards, Explainable AI.

ACM Reference Format: Aaditya Borse. 2025. Designing Transparent and Adaptive Interfaces for AI Tutors: A Human–Computer Interaction Approach. Irvine, CA, USA.

1 Introduction

Effective studying remains a central component of secondary education, yet students in grades 6–12 frequently struggle to manage their learning in systematic and productive ways. Increasing homework lengths, big exams, and fast-paced school environments make it difficult for students to identify which topics to prioritize, how to regulate their pacing, and how to monitor their own progress. Research in self-regulated learning and human-centered learning analytics shows that without structured guidance, students often adopt inefficient learning strategies, overlook misconceptions, and fail to target their areas of weakness effectively [2, 4].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SWE 233 Project Prototype, UC Irvine

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Intelligent tutoring systems (ITS) have demonstrated that adaptive feedback and mixed-initiative dialogue can meaningfully improve learning outcomes compared to static instruction [6, 9]. However, most traditional ITS offer limited transparency into why the system recommends certain steps, provide minimal learner control, and do not integrate analytics directly into the learning activity. These limitations constrain learners’ ability to interpret feedback [9].

Recent advances in intelligent systems and large language models (LLMs) create new opportunities for tutoring systems that combine conversational guidance with adaptive recommendations. LLM-powered tutors can parse open-ended student input and generate contextually rich explanations, but they also introduce challenges: opaque reasoning processes, inconsistent feedback, and a lack of mechanisms for learner agency [3, 7, 13]. Addressing these challenges requires rethinking adaptation.

This work introduces **LEO** (Learning Education Operative), a student-facing study companion that emphasizes transparent reasoning, interactive visualizations, and learner control. Rather than optimizing backend prediction models, LEO focuses on making the system’s adaptive decisions visible and give a clearer understanding of how the system responds to their actions.

This research aims to:

- (1) Evaluate how integrating adaptive feedback and conversational guidance affects student engagement, cognitive load, and focus.
- (2) Investigate how transparent, interactive visualizations support learner autonomy and self-regulated learning.

To frame this work, we pose three design-focused research questions:

- (1) **Transparency and Explanation:** How can the tutor interface expose the system’s reasoning and adaptation so that students understand why particular hints, examples, or pacing adjustments are suggested?
- (2) **Human–Intelligent System Interaction:** How should conversational tutoring and learning analytics be combined to support effective collaboration between students and intelligent systems?
- (3) **Agency and Engagement:** Which interface features strengthen students’ sense of control and comprehension of system logic?

2 Related Work

This research builds on work in intelligent tutoring systems, learning analytics, and human–AI interaction. We focus on three strands of literature that directly inform the design of LEO: (1) modern intelligent tutoring and LLM-based adaptive systems; (2) learning

analytics and actionable visualizations; and (3) human-centered explainability and interface transparency.

2.1 Intelligent Tutoring and Adaptive Systems

Modern intelligent tutoring systems (ITS) have demonstrated that personalized scaffolding and mixed-initiative dialogue can produce substantial learning gains over static instruction. Meta-analytic reviews show that adaptive tutors consistently outperform traditional methods [9].

However, most traditional ITS rely on rigid student models, hand-authored rules, or narrowly scoped domain representations that limit adaptability and interpretability. Recent work examining LLM-powered tutors [3, 5, 7, 13] shows that while these models can generate rich explanations and respond flexibly to student queries, their reasoning often remains inconsistent or misaligned with learner needs.

LEO positions itself by emphasizing transparent and controllable adaptation. Rather than treating intelligence as an internal black box, LEO puts the interface as the place where system decisions become interpretable and actionable.

2.2 Learning Analytics and Visualizations

Learning analytics dashboards have shown promise in supporting reflection, metacognition, and study planning when designed with learners’ interpretive needs in mind [11]. Students benefit from visualizations that contextualize progress, highlight misconceptions, and directly suggest actionable next steps. Human-centered learning analytics frameworks argue that dashboards should be participatory and linked to long-term goals. [4, 12].

Despite these advances, many analytics tools suffer from what is often termed the *analytics–practice gap*: dashboards summarize performance but remain disconnected from the immediate learning activity. Students must manually translate insights into next actions, which increases cognitive load and reduces the utility of analytics for real-time decision-making.

LEO addresses this gap by embedding mastery visualizations directly into the tutoring flow. When students view their progress or misconceptions, they can immediately initiate targeted practice, review worked examples, or request conceptual explanations. This design transforms dashboards into active learning interfaces that help reduce wasted time.

2.3 Human-Centered and Explainable Responses

Recent work in explainable AI and educational dialogue systems has emphasized the importance of making system reasoning transparent and user-aligned. Delikoura et al. propose dialogical LLMs that incorporate human preference graphs to improve interpretability and pedagogical consistency [1]. Complementary tools such as StuGPTViz visualize student–AI interaction patterns to help stakeholders understand how learners engage with conversational systems [14].

However, these systems often remain descriptive rather than interactive: they explain how conversations unfold but do not allow learners to query the system’s reasoning or adjust its adaptive

behavior. Reviews of LLM tutors consistently highlight the need for controllable and evidence-grounded feedback [3, 13].

HCI research on transparency design provides guidance for creating such systems. Effective explanations should be concise, causally framed, and directly tied to user context [1]. LEO operationalizes these principles by attaching short rationales to every adaptive decision, linking each suggestion to visible evidence (e.g., recent errors, mastery estimates), and providing user-adjustable controls for pacing, hint density, and focus mode.

Through this combination tactics, LEO extends prior work by positioning learners not merely as recipients of feedback but as active participants in shaping how the intelligent system guides their study process.

3 Proposed Solution

Our proposed system, LEO (Learning Education Operative), is a student-facing study companion that pairs a conversational tutor with an integrated learning dashboard. LEO does not pursue new backend learning algorithms; instead, its contribution is an interface-mediated approach to adaptability, making system decisions visible. In doing so, LEO addresses three recurring weaknesses in prior systems: poor adaptation logic, disconnects between analytics and practice, and limited learner control [1, 2, 11].

The project focuses on secondary-school learners (grades 6–12) working in individual study contexts such as homework, review, and self-paced practice. LEO specifically targets common, measurable curricular tasks across subjects, with the initial prototype centered on middle/high school mathematics. Typical users are students who routinely engage in problem-solving, seek step-by-step support, and use the system as a Q&A aid for homework or formative test preparation. Scoping the system in this way enables a focused design and evaluation plan while retaining the architecture necessary to generalize to other K–12 subjects.

3.1 System overview

Figure 1 structures the system overview into three coordinated components that exchange information continuously. The diagram illustrates how student actions enter the Intelligent System, how they update mastery and activity signals in the Data Sources, and how these changes propagate back to the UI panels. This closed-loop interaction is central to our research focus on transparency and user control. Each subsystem described in this section corresponds to elements in the architecture: student inputs drive the Intelligent System’s inferences, which then drive adaptive tutoring responses and dashboard updates.

The chat interface supports question answering, worked examples, and brief diagnostic interactions. It uses guided prompts to structure responses when students show uncertainty or repeated error patterns, drawing on principles established in prior ITS research [6, 9]. When the tutor issues a suggestion, it attaches a short, plain rationale tied to observable student activity mirroring the “Intelligent System → UI Layer” path in Figure 1. This approach also responds to recent findings that LLM-generated feedback is often inconsistent or insufficiently contextualized [3, 7, 13]. The dashboard visualizes topic-level mastery, recent activity, and pacing patterns. In the architecture diagram, this corresponds to the

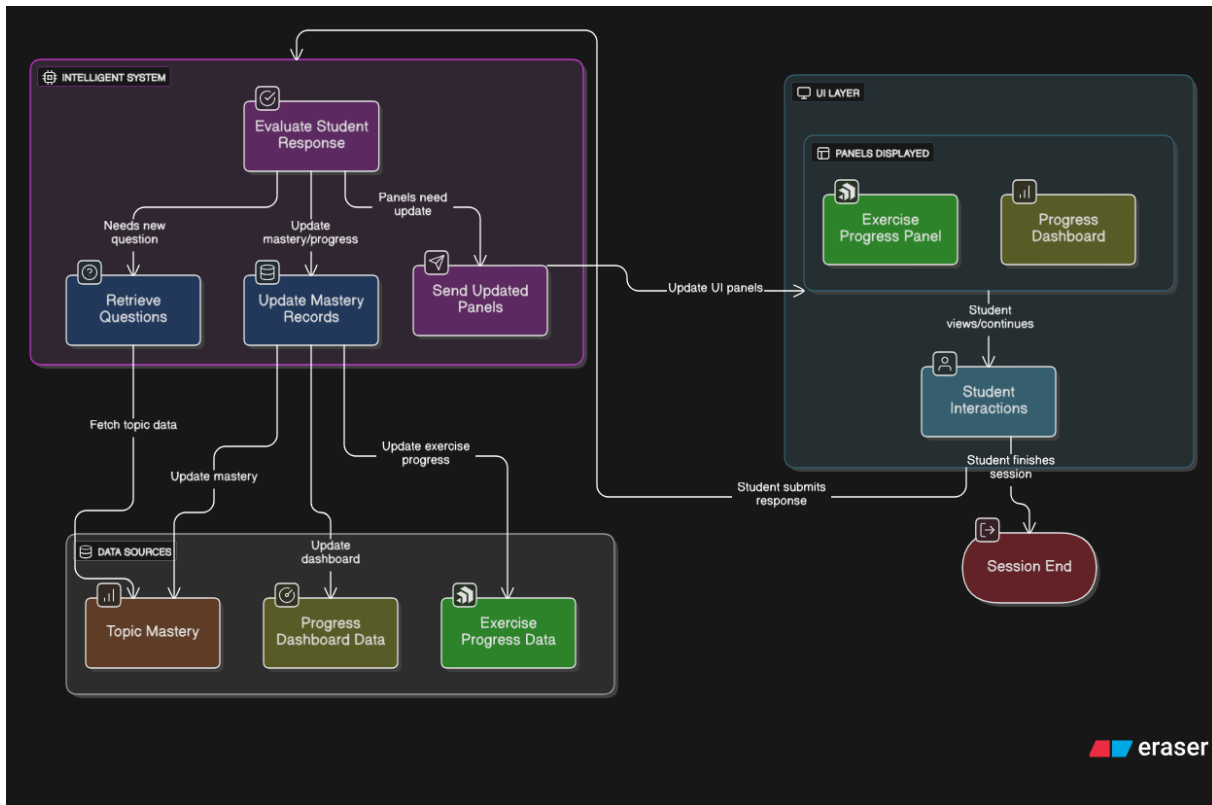


Figure 1: System architecture for LEO, showing how student input flows into the Intelligent System, updates Data Sources, and propagates to the UI Layer.

“UI Panels” that update whenever the Intelligent System modifies mastery records in the data layer. Visual elements are interactive: clicking a low-mastery bar launches a context-specific dialogue that reviews mistakes and offers targeted practice. The design emphasizes interpretable visualizations to support learner reflection, building previous HCI frameworks [4, 11].

The transparency layer makes explicit the “Intelligent System → Explanation” arrows in Figure 1. Every recommendation links to an explanation pane that surfaces the evidence and heuristic behind the suggestion. Explanations follow design principles that emphasize concise, actionable reasoning [1]. Students may accept a recommendation, request more scaffolding, or adjust controls. These controls correspond to modifiable parameters in the Intelligent System and enter the same adaptive loop depicted in the architecture. Recent work on dialogical and interpretable LLMs motivates this layer [1].

3.2 Adaptive interaction design

The adaptive loop presented in Figure 1 is implemented directly in LEO’s interaction model. Student actions update a model that expresses estimated concept mastery and short-term engagement. The dashboard displays the most relevant aspects of that model, and the chat uses the same model to choose next steps. This means each

student action produces immediate changes in both the visual summary and the system’s subsequent dialogue, corresponding to the two-way arrows between “Student Response,” “Mastery Records,” and “UI Panels.”

The interface signals changes with confidence bars, inline rationale links, and optional pacing sliders. These cues support actionable understanding, aligning with modern design recommendations in transparency and learning analytics [4]. To manage cognitive load, feedback is broken into concise, sequential steps.

3.3 How LEO improves on past oversights

LEO addresses three specific gaps identified in the literature, each reflected in the architectural choices shown in Figure 1:

Opaque adaptation. Traditional tutors adapt behavior but do not reveal their criteria. LEO makes adaptation legible through concise rationales and linked evidence, reducing blind trust and enabling informed acceptance or rejection [1].

Analytics–practice gap. Dashboards frequently require manual interpretation and separate reflection from practice. LEO links visuals to action: clicking a visualization opens a tailored tutoring episode, unifying reflection and practice within the same adaptive loop [10, 11]. This corresponds to the direct “UI Panels Intelligent System” connection in the architecture.

Limited learner control. Many conversational tutors drive interaction without giving learners control over pace or preference. LEO provides direct controls that map to editable parameters in the Intelligent System. These changes immediately propagate through the adaptive loop and update both the UI and mastery representations [2, 7].

3.4 Design novelty and expected benefits

LEO's novelty lies in prioritizing low-friction workflows, where moving from error analysis to targeted practice requires minimal interaction. The architectural diagram underscores that every cycle from student input to UI update carries both content and an explanation. By embedding transparent rationales into both chat and dashboard, LEO reduces the interpretive work learners must perform and makes adaptive recommendations actionable. We expect this to improve students' ability to prioritize study topics, increase acceptance of helpful suggestions, and reduce reliance on confident but incorrect answers [3, 8, 13].

4 System Design

As shown in Figure 1, LEO's system design follows a three-stage adaptive loop: **Input Understanding**, **Adaptive Reasoning**, and **Interface Communication**. This loop governs how student inputs propagate through the system, update mastery representations, and return as actionable UI changes. The arrows in the architecture diagram represent the continuous updating process. The architectural structure in Figure 1 frames the interaction model used throughout the system: student actions flow into the Intelligent System for reasoning, and the resulting updates flow back into UI elements.

To illustrate how this architecture works in real interaction, Figure 2 shows the main LEO practice workspace. In this workspace, the left panel reflects student input and step-by-step reasoning, while the right panel displays adaptive feedback, concept mastery indicators, and pacing guidance driven by the Intelligent System. Together, Figures 1 and 2 visualize both the high-level system logic and its concrete instantiation in the interface.

4.1 System Architecture Overview

Figure 1 outlines the system pipeline that processes student interactions. The flow from student input, through the Intelligent System, and back to the UI panels represents the core mechanism that enables LEO's adaptive and explainable behavior. The practice interface in Figure 2 is the surface-level manifestation of this deeper architectural process. The system operates as follows:

- (1) **Input Understanding.** Students submit queries or partial solutions via an event-driven POST request. The payload includes the raw equation, the student's recent error history, and their current mastery state. **Rule-based parsing** first filters for common syntax errors before passing the context to the reasoning engine.
- (2) **Adaptive Reasoning.** The core reasoning layer utilizes the **Gemini 3 API** for low-latency inference. It operates on a hybrid logic model: deterministic rules handle structural scaffolding, while the LLM generates explanatory reasoning. The system targets a <500ms latency to ensure real-time responsiveness. This layer generates three adaptive outputs:

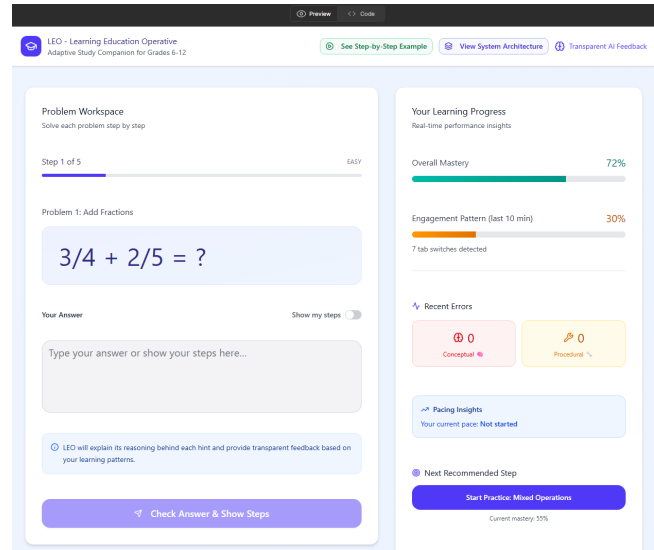


Figure 2: The main LEO practice workspace. Students solve problems in the left workspace and mastery indicators appear in the right panel.

- **Diagnostic Feedback:** Identifies the type of error or misconception.
 - **Conceptual Hint:** Connects the problem to prior knowledge or relevant principles.
 - **Explanatory Prompt:** Encourages reflection and self-correction.
- (3) **Interface Communication.** The API returns a structured JSON object which the **React frontend** parses. The UI layer decouples the "Feedback" from the "Reasoning," ensuring that the explanation (the "Why this feedback?" component) is only revealed when requested by the user, managing cognitive load while maintaining transparency.

The system maintains a coupled feedback loop: student actions instantly modify the user model, which then immediately informs both adaptive reasoning and interface visualization. This reflects the bidirectional arrows in Figure 1 that represent continuous updating between the Intelligent System, the Knowledge Base, and the UI Layer. The design ensures that reflection, feedback, and action occur seamlessly in a single interaction flow.

4.2 Example Interaction Scenario

The scenario below demonstrates how the architectural flow in Figure 1 and the interface structure in Figure 2 work together in practice.

Consider a 7th-grade student solving a fraction addition problem:

- (1) The student submits the answer $\frac{3}{4} + \frac{2}{5} = \frac{5}{9}$ in the workspace.
- (2) Input Understanding parses the answer, detects a common misconception in finding a common denominator, and updates the student model to reflect this error type.
- (3) Adaptive Reasoning generates:
 - Diagnostic: "You added the numerators and denominators directly."

- Conceptual Hint: “Remember to find a common denominator before adding fractions.”
 - Explanatory Prompt: “Try rewriting $\frac{3}{4}$ and $\frac{2}{5}$ with the least common denominator and add again.”
- (4) Interface Communication renders feedback cards with a “Why this feedback?” link. Clicking it reveals the rationale: the system matched the student’s error pattern to the common-denominator rule and used past activity to tailor the hint.
 - (5) The student revises their answer, and the cycle repeats, updating mastery metrics in the progress dashboard.

This scenario illustrates how LEO makes adaptive feedback meaningful for students while abstracting the underlying LLM computations [8, 13]. It also mirrors the adaptive loop depicted in Figure 1, demonstrating how each layer of the system contributes to transparency, self-regulation, and student agency.

5 Interface Design

As shown in Figure 3, LEO’s interface is designed to align directly with the adaptive loop in Figure 1, where student actions update the Intelligent System and trigger corresponding UI changes.

The interface addresses limitations in prior ITS and LLM-powered tutors [1, 6, 14] by simplifying system reasoning and giving learners meaningful control over the tutoring process. The interface is organized into three interconnected components that form the student’s primary learning environment. These components mirror the “UI Layer” in Figure 1 and instantiate its feedback pathways.

1. Problem Workspace. The *Problem Workspace* is the primary focus of student activity and corresponds to the left pane in Figure 2. Students enter solutions, show intermediate steps, ask questions, or view worked examples using text, symbolic notation, or voice input. This supports proven studying techniques, addressing findings that students often adopt inefficient strategies without structured guidance [6]. The workspace is tightly integrated with the architecture’s “Input Understanding” loop.

2. Dynamic Feedback Panel. The *Dynamic Feedback Panel* complements the workspace by presenting diagnostic messages, conceptual hints, and explanatory prompts as interactive cards. Each card includes a “Why this feedback?” link that reveals evidence and reasoning behind the suggestion, which relates to the explanation pathway shown in Figure 1. These rationales are derived from student activity such as recent attempts, error patterns, or mastery metrics. This design directly addresses the opacity commonly observed in LLM tutors [1, 14], while reinforcing conceptual understanding.

3. Progress Visualization Dashboard. The *Progress Visualization Dashboard*, depicted in Figure 3, summarizes concept mastery, pacing patterns, and error types (procedural vs. conceptual). These representations serve as the primary visualization of the student model stored in the data layer of Figure 1. The visualizations are interactive, allowing students to select low-mastery concepts to trigger targeted practice exercises, example walkthroughs, or short instructional videos. By linking analytics to concrete remediation steps, the dashboard uses analytics principles described in recent studies [4, 11].

All interface components are tightly synchronized through the underlying student model. Updates in the Problem Workspace

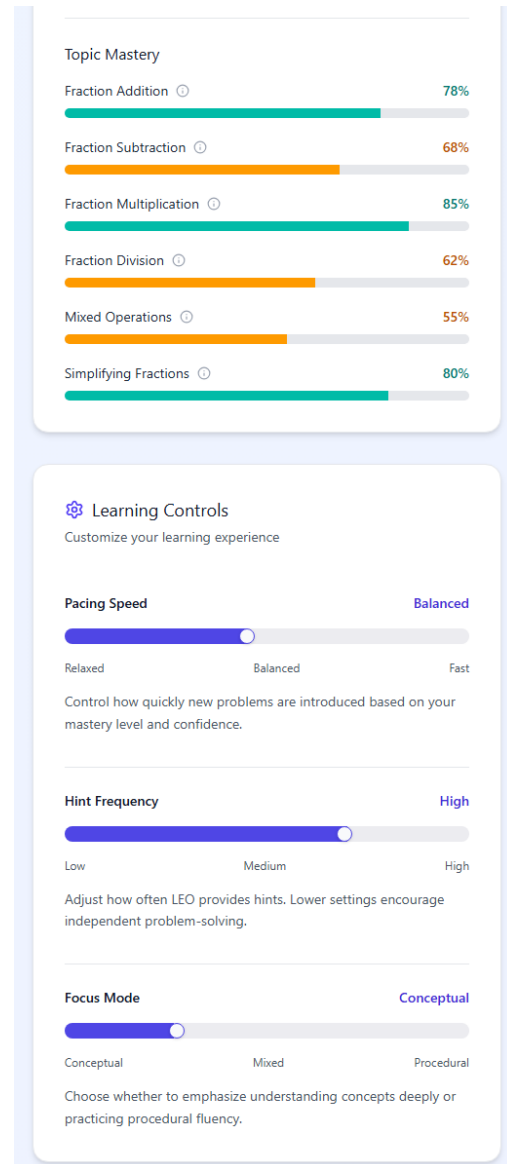


Figure 3: LEO’s interactive dashboard and learning controls, showing mastery indicators, pacing adjustments, hint-frequency tuning, and focus mode controls.

immediately refresh feedback and dashboard visualizations, completing the adaptive loop depicted in Figure 1. This ensures that reflection, feedback, and action occur seamlessly within a single interaction context.

A limited set of controls allows learners to modulate tutoring behavior without needing to understand the underlying LLM or inference rules. These controls correspond to adjustable parameters within the Intelligent System that flow back into the UI Layer, as illustrated in the architecture diagram. This design promotes learner agency and trust by being open and easy to use [1].

6 Technical Implementation Plan

The implementation of LEO is scoped for a four-week development cycle, focusing on a functional vertical prototype for Algebra I. The system architecture relies on a decoupled client-server model to ensure responsiveness and modularity.

6.1 Technology Stack

The core technologies selected for the prototype are:

- **Backend:** Python with Flask. This framework was chosen for its lightweight nature and seamless integration with AI libraries. It serves as the API gateway between the client and the inference engine.
- **Frontend:** React.js. A component-based library allows us to manage the complex state of the "Problem Workspace" and "Dashboard" independently.
- **Data Storage:** Firebase Firestore. A NoSQL document store is used to log unstructured interaction traces and JSON-based mastery records in real-time.
- **Inference Engine:** Google Gemini 3 API. We utilize the Gemini 3 Flash variant to prioritize low latency (<500ms) for real-time chat, using a temperature setting of 0.3 to reduce hallucination risks while maintaining conversational fluency.

6.2 Component Interaction and Data Flow

To address the "black box" limitations of prior systems, LEO implements a structured request-response cycle that separates pedagogical guidance from system reasoning. This flow operationalizes the high-level pathways depicted in Figure 1.

1. Request Triggering (Student Submits Response): Interactions follow an event-driven architecture. Corresponding to the *Student submits response* arrow in the diagram, when a learner enters a solution in the React frontend, an asynchronous POST request is sent to the Flask backend. The payload aggregates three context vectors to minimize state lookups:

- **Immediate Context:** The current equation or natural language query.
- **Historical Context:** The last 3 turns of dialogue and session-specific error tags.
- **User State:** A JSON snapshot of the student's current mastery estimation, derived from the *Progress Dashboard Data* shown in the diagram.

2. Processing and Inference (Evaluate Student Response): Inside the *Evaluate Student Response* module, the Flask server constructs a dynamic system prompt. To prevent hallucination, we decouple verification from explanation: a symbolic math library (SymPy) first deterministically verifies the answer's correctness. The Gemini 3 API is then invoked solely to generate the *scaffolding and explanation*.

We utilize structured output enforcement (JSON mode) to ensure the model returns a distinct "message" field (the pedagogical feedback) and a "rationale" field (the evidence-based logic).

3. Parsing and Rendering (Send Updated Panels): Following the *Update UI panels* flow, the frontend parses the returned JSON object to refresh the *Panels Displayed*. The "message" is rendered immediately in the chat stream. Crucially, the "rationale" populates

the hidden "Why this feedback?" component within the *Exercise Progress Panel*. This separation ensures that while system reasoning is transparent, it does not obstruct the primary learning workflow.

7 Evaluation Plan

To assess whether LEO effectively addresses the problems of opaque adaptation and limited learner control, we propose a comprehensive mixed-methods evaluation. This study is designed to measure learning outcomes through standardized math assessments and behavioral engagement logs.

7.1 Research Questions

Our inquiry focuses on three specific questions that map to the design goals identified in Section 3:

- **RQ1 (Transparency & Trust):** Does exposing the rationale behind adaptive feedback ("Why this feedback?") increase student trust and perceived understanding of the system's logic compared to a black-box baseline?
- **RQ2 (Agency & Persistence):** Does providing explicit controls for pacing and hint density increase task persistence (measured by problem attempts) and reduce hint dependency?
- **RQ3 (Learning Efficiency):** Does the combination of transparent rationales and interactive dashboards lead to more efficient remediation (measured by time-to-mastery) compared to standard immediate-feedback tutors?

7.2 Methodology

We propose a **within-subjects user study** with $N = 40$ participants (grades 7–9). A within-subjects design was selected to control for high individual variance in prior mathematical knowledge, ensuring that performance differences can be attributed to the interface conditions rather than student ability.

7.2.1 Conditions. Each participant will interact with two distinct system configurations in a counterbalanced order to negate learning effects:

- (1) **Control Condition (Black Box):** A standard AI tutor where the "Why this feedback?" explanations are hidden, and the "Learning Controls" (pacing/hint density) are disabled. The underlying adaptive logic remains active but invisible.
- (2) **Experimental Condition (LEO):** The full system with visible reasoning traces, interactive dashboards, and active learning controls.

7.2.2 Task & Procedure. The study will be conducted remotely via a web browser, lasting approximately 60 minutes.

- (1) **Pre-Screening:** Participants engage in a 5-minute calibration task using items from the **NAEP Questions Tool** (National Assessment of Educational Progress) to establish a baseline Algebra proficiency.
- (2) **Interaction Phase 1 (20 min):** Participants solve a set of 10 adaptive algebra problems using their first assigned condition (Randomized Group A or B).
- (3) **Washout Period (10 min):** A mandatory break with a non-mathematical distractor task to minimize cognitive fatigue and carryover effects.

- (4) **Interaction Phase 2 (20 min):** Participants solve an isomorphic problem set (equal difficulty) using the second condition.
- (5) **Post-Test & Interview:** A parallel assessment to measure immediate learning gains, followed by a semi-structured interview.

7.3 Metrics and Data Collection

We employ a triangulation of quantitative logs, standardized test scores, and qualitative feedback.

1. Math Proficiency & Accuracy (RQ3). We rely on validated items from the ****NAEP Questions Tool**** to measure skill acquisition.

- **Normalized Learning Gain:** calculated as $\frac{\text{PostTest} - \text{PreTest}}{100 - \text{PreTest}}$.
- **First-Attempt Accuracy:** The percentage of problems solved correctly on the first attempt during the interaction phases.

2. Behavioral Process Metrics (RQ2). System logs will capture behavioral data to serve as a proxy for engagement and agency, replacing subjective workload surveys.

- **Hint Dependency Ratio:** The average number of hints requested per problem solved.
- **Action Depth:** The frequency of clicks on rationale links and adjustments to pacing sliders.
- **Feedback Acceptance Rate:** The percentage of AI suggestions that are followed by a correct student action [8].

3. Trust & Explainability (RQ1). To rigorously measure trust, we will use the Trust in Automation Scale (TIAS) adapted for educational AI contexts. This allows us to quantify if the "Why this feedback?" feature specifically impacts the "Understanding" and "Reliability" sub-scales of trust [1].

8 Threats to Validity

While the proposed within-subjects design controls for many confounding variables, several threats to validity remain due to the prototype nature of the system and the constraints of the evaluation timeline.

8.1 Internal Validity

Internal validity concerns the extent to which the observed effects can be attributed to the LEO interface rather than extraneous factors.

- **Carryover and Learning Effects:** In a within-subjects design, participation in the first condition may influence performance in the second. While we employ strict counterbalancing and isomorphic problem sets, it is possible that the *strategy* of checking for hints in the Experimental condition could carry over to the Control condition, potentially dampening the measured difference.
- **Fatigue and Maturation:** A 60-minute session involving algebraic problem-solving is cognitively demanding. Participants may perform worse in the second block simply due to mental exhaustion. We mitigate this with a mandatory 10-minute "washout" period involving a non-mathematical distractor task, but fatigue remains a potential confound.

- **Selection Bias:** Participants recruited from voluntary STEM after-school programs may possess higher intrinsic motivation or baseline self-regulation skills than the general student population, potentially skewing engagement metrics.

8.2 External Validity

External validity concerns the generalizability of our findings to broader contexts.

- **Ecological Validity:** The study occurs in a controlled, time-boxed environment which differs from the "messy" reality of unsupervised homework. In a real-world setting, students often multi-task or have intermittent focus; our continuous 20-minute sessions may overestimate the sustained attention students would actually dedicate to reading rationales.
- **Domain Specificity:** Linear Equations are highly structured with clear deterministic rules. The effectiveness of the "Why this feedback?" feature may not generalize to ill-structured domains (e.g., Literature or History) where AI reasoning is inherently more probabilistic and subjective.

8.3 Construct Validity

Construct validity examines whether our metrics accurately measure the theoretical concepts of "Trust" and "Agency."

- **Social Desirability Bias:** Students may rate the experimental system higher on "Helpfulness" or "Trust" surveys simply because they perceive it as the "newer" or "better" technology, rather than based on genuine utility. We attempt to control for this by triangulating survey data with behavioral logs (e.g., actual hint usage rates).
- **Novelty Effect:** Increased engagement with the pacing sliders and dashboards may result from the novelty of the interactive elements rather than their pedagogical value. A longitudinal study would be required to determine if these effects persist once the novelty fades.

8.4 Conclusion Validity

Conclusion validity addresses the statistical power and reliability of our results.

- **Sample Size and Power:** With $N = 40$, the study is powered to detect medium-to-large effect sizes. Subtle interactions—such as the specific impact of transparency on high-performing vs. low-performing students—may be statistically undetectable without a larger sample.
- **Measure Reliability:** While NAEP items are standardized, the mapping of "learning gain" over a short 20-minute intervention is noisy. Random heterogeneity in student background (grades 7 vs. 9) may introduce variance that obscures the treatment effect.

9 Conclusion

LEO represents a shift in intelligent tutoring systems from opaque automation to transparent collaboration. By exposing the "black box" of AI reasoning and giving students control over their learning environment, LEO aims to foster not just content mastery, but metacognitive awareness. The system architecture, leveraging the

Gemini 3 API for reasoning and a React-based dashboard for transparency, provides a robust foundation for the proposed evaluation. If the pilot study confirms that transparency reduces frustration and builds trust, LEO could serve as a model for the next generation of human-centered educational AI.

References

- [1] Saleema Amershi, Daniel S. Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi T. Iqbal, Paul N. Bennett, Kori Inkpen, et al. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13. doi:10.1145/3290605.3300233
- [2] Roger Azevedo, François Bouchet, Melissa Duffy, Jason Harley, Michelle Taub, Gregory Trevors, and Michael Wiedbusch. 2022. Lessons Learned and Future Directions of MetaTutor: Leveraging Multichannel Data to Scaffold Self-Regulated Learning With an Intelligent Tutoring System. *Frontiers in Psychology* 13 (2022). doi:10.3389/fpsyg.2022.813632
- [3] Wei Dai, Yi-Shan Tsai, Jionghao Lin, Dragan Gašević, and Guanliang Chen. 2024. Assessing the proficiency of large language models in automatic feedback generation: An evaluation study. *Computers and Education: Artificial Intelligence* 6 (2024), 100299. doi:10.1016/j.caeai.2024.100110
- [4] Yannis Dimitriadis, Ricardo Martinez-Maldonado, and Simon Buckingham Shum. 2021. Human-centered learning analytics: A conceptual framework. *Journal of Learning Analytics* 8, 2 (2021), 1–20. <https://learning-analytics.info/index.php/JLA/article/view/6627>
- [5] I. Estévez-Ayres et al. 2024. Evaluation of LLM Tools for Feedback Generation in a Course on Concurrent Programming. *International Journal of Artificial Intelligence in Education* (2024). doi:10.1007/s40593-024-00406-0
- [6] Arthur C. Graesser, Shulan Lu, George T. Jackson, Heather H. Mitchell, Mathew Ventura, Andrew Olney, and Max M. Louwerse. 2004. AutoTutor: A tutor with dialogue in natural language. *Behavior Research Methods, Instruments, & Computers* 36, 2 (2004), 180–192. doi:10.3758/BF03195563
- [7] Wayne Holmes and Ilkka Tuomi. 2022. Artificial intelligence in education: Promises and implications for teaching and learning. *Computers and Education: Artificial Intelligence* 3 (2022), 100050. doi:10.1016/j.caeai.2022.100050
- [8] Charles Koutchme et al. 2024. Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4--As-A-Judge. In *Proceedings of the 2024 Conference on Innovation and Technology in Computer Science Education (ITiCSE)*. doi:10.1145/3649217.3653612
- [9] James A. Kulik and John D. Fletcher. 2016. Effectiveness of intelligent tutoring systems: A meta-analytic review. *Review of Educational Research* 86, 1 (2016), 42–78. doi:10.3102/0034654315581420
- [10] Chien-Chang Lin, Chia-Chen Huang, and Shu-Ching Lu. 2023. Artificial intelligence in intelligent tutoring systems toward sustainable education: a systematic review. *Smart Learning Environments* 10 (2023). doi:10.1186/s40561-023-00260-y
- [11] Bo Pei, Ying Cheng, Eva Dziadula, G. Alex Ambrose, Wanli Xing, and Jie Lu. 2024. LearningViz: A dashboard for visualizing, analyzing and closing learning performance gaps — a case study approach. *Smart Learning Environments* 11 (2024). doi:10.1186/s40561-024-00346-1
- [12] S. Z. Salas-Pilco, K. Xiao, and X. Hu. 2022. Artificial Intelligence and Learning Analytics in Teacher Education: A Systematic Review. *Education Sciences* 12, 8 (2022). doi:10.3390/educsci12080569
- [13] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research (ICER)*. 27–39. doi:10.1145/3501385.3543957
- [14] Jorge C. Valverde-Rebaza, A. González, O. Navarro-Hinojosa, and J. Noguez. 2024. Advanced large language models and visualization tools for data analytics learning. *Frontiers in Education* 9 (2024). doi:10.3389/feduc.2024.1418006
- (3) **Word Problem:** A rental car company charges \$25 per day plus \$0.10 per mile. If the total cost for one day was \$45, how many miles were driven? Write the equation and solve.
- (4) **Fractional Coefficients:** Solve for y : $\frac{1}{2}y - 4 = 8$.
- (5) **Conceptual Understanding:** Which of the following equations represents a line that passes through the point (0, 5) and has a slope of 2?
 - (a) $y = 5x + 2$
 - (b) $y = 2x + 5$
 - (c) $y = 2x - 5$
 - (d) $y = 5x - 2$

A Appendix: Sample NAEP Evaluation Items

The following 5 items represent **Problem Set A** (Linear Equations), derived from the NAEP 8th-Grade Mathematics Assessment. These items serve as the primary instrument for measuring Learning Gains (RQ3).

- (1) **Basic Solve:** Solve for x : $3x + 5 = 20$.
- (2) **Variables on Both Sides:** What is the value of x if $4x + 6 = 2x + 16$?